

ASYMPTOTIC NOTATION

Asymptotic Notations

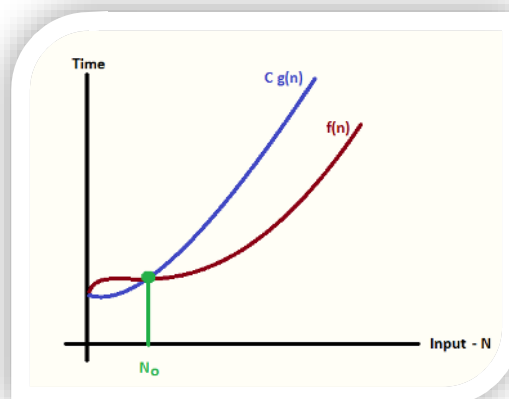
- **Asymptotic notations** are the mathematical notations used to describe the running time of an algorithm when the input tends towards a particular value or a limiting value.
- **For example:** In **bubble sort**, when the input array is already sorted, the time taken by the algorithm is linear i.e. **the best case**.
- But, when the input array is in reverse condition, the algorithm takes the maximum time (quadratic) to sort the elements i.e. **the worst case**.
- When the input array is neither sorted nor in reverse order, then it takes average time. These durations are denoted using **asymptotic notations**.

There are mainly **three** asymptotic notations:

- **Big-O notation**
- **Omega notation**
- **Theta notation**

Big-O Notation (O-notation)

- ✓ Big-O notation represents the upper bound of the running time of an algorithm.
- ✓ Thus, it gives the worst-case complexity of an algorithm.



$$O(g(n)) = \{ f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq f(n) \leq cg(n) \text{ for all } n \geq n_0 \}$$

Example:

Consider the following $f(n)$ and $g(n)$...

$$f(n) = 3n + 2$$

$$g(n) = n$$

If we want to represent $f(n)$ as $O(g(n))$ then it must satisfy $f(n) \leq C g(n)$ for all values of $C > 0$ and $n \geq 1$

$$f(n) \leq C g(n)$$

$$\Rightarrow 3n + 2 \leq C n$$

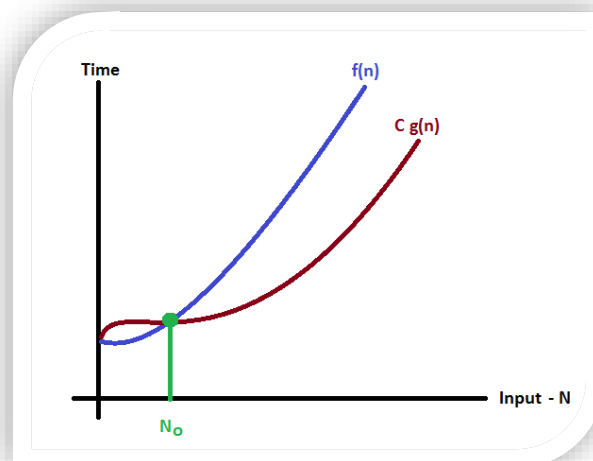
Above condition is always TRUE for all values of $C = 4$ and $n \geq 2$.

By using Big - Oh notation we can represent the time complexity as follows... $3n + 2 = O(n)$.

Omega Notation (Ω -notation)

Omega notation represents the lower bound of the running time of an algorithm.

Thus, it provides the best case complexity of an algorithm.



$$\Omega(g(n)) = \{ f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0 \}$$

Example:

Consider the following $f(n)$ and $g(n)$...

$$f(n) = 3n + 2$$

$$g(n) = n$$

If we want to represent $f(n)$ as $\Omega(g(n))$ then it must satisfy $f(n) \geq C g(n)$ for all values of $C > 0$ and $n \geq 1$

$$f(n) \geq C g(n)$$

$$\Rightarrow 3n + 2 \geq C n$$

Above condition is always TRUE for all values of $C = 1$ and $n \geq 1$.

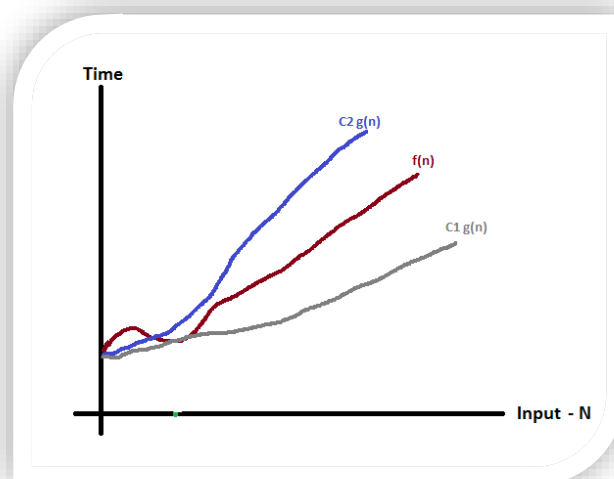
By using Big - Omega notation we can represent the time complexity as follows...

$$3n + 2 = \Omega(n)$$

Theta Notation (Θ -notation)

Theta notation encloses the function from above and below.

Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm.



$$\Theta(g(n)) = \{ f(n) : \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for all } n \geq n_0 \}$$

Consider the following **$f(n)$** and **$g(n)$** ...

$$f(n) = 3n + 2$$

$$g(n) = n$$

If we want to represent **$f(n)$** as **$\Theta(g(n))$** then it must satisfy **$C1 g(n) \leq f(n) \leq C2 g(n)$** for all values of **$C1 > 0$** , **$C2 > 0$** and **$n \geq 1$**

$$C1 g(n) \leq f(n) \leq C2 g(n)$$

$$\Rightarrow \mathbf{C1 n \leq 3n + 2 \leq C2 n}$$

Above condition is always TRUE for all values of **$C1 = 1$** , **$C2 = 4$** and **$n \geq 2$** .

By using Big - Theta notation we can represent the time complexity as follows...

$$3n + 2 = \Theta(n)$$